

Tasks 1–4 are walked through step-by-step in [z3]. Follow the code listings and explanations in that section to complete the FOL domain setup (task 1), quantified physics rules (task 2), manual reasoning walkthrough (task 3), and full agent implementation (task 4). The manual reasoning results should match those of the propositional agent from [k7]; the agent should retrieve the package and exit with the same step count and reward.

5. Domain closure investigation

Removing the domain closure axiom (`ForAll(L, Or([L == loc[...] ...]))` and `Distinct(...)`) from `build_warehouse_kb_fol()` causes the *elimination-based* entailment queries in the manual reasoning trace to fail, while queries derived directly from *negative* percepts still succeed.

Queries that still work: After TELLing no creaking and no rumbling at `loc1`, the entailments `Adjacent(loc1, loc2)` and `Adjacent(loc2, loc3)` still hold. This is because the reasoning relies on ground adjacency facts: `Adjacent(loc1, loc2)` is asserted, and `Adjacent(loc2, loc3)` forces `Adjacent(loc1, loc3)` via the quantified rule instantiated at `loc2`. No phantom location is involved—the conclusion follows directly from a ground fact and a universal instantiation. Similarly, after three percepts, `Adjacent(loc1, loc2)` is still entailed because `Adjacent(loc1, loc2)` follows from no creaking at `loc1` (via the ground fact `Adjacent(loc1, loc2)`) and `Adjacent(loc2, loc3)` follows from no rumbling at `loc2` (via `Adjacent(loc2, loc3)`).

Queries that fail: After TELLing creaking at `loc1`, the quantified rule requires:

With domain closure, this existential can only be witnessed by real grid squares. Since `Adjacent(loc1, loc2)` is known, the solver concludes `Adjacent(loc1, loc2)`. Without domain closure, `DeclareSort('Location')` creates an *uninterpreted* sort, and Z3 is free to invent a phantom location `loc4`. The closed-world adjacency facts only cover pairs of *known* grid constants; `Adjacent(loc1, loc4)` is unconstrained, so the solver can set it to true and let `Adjacent(loc1, loc2)` witness the existential. In this model, no real grid square needs to be damaged.

As a result, the elimination chain breaks: after all three percepts, the solver cannot conclude `Adjacent(loc1, loc2)` (damage at `loc1`), `Adjacent(loc2, loc3)` (forklift at `loc2`), or that `Adjacent(loc1, loc3)` and `Adjacent(loc2, loc3)` are dangerous. Any query that requires process-of-elimination—ruling out all alternatives to pin down a specific location—fails because phantom locations always offer an escape.

6. Reflection

The propositional encoding requires `grid_size * grid_size` grounded biconditionals (one per square per predicate type), totaling `3 * grid_size * grid_size` physics sentences for an `grid_size` grid. The FOL encoding uses exactly 3 quantified sentences for the physics rules regardless of grid size, though it still needs `grid_size * grid_size` closed-world adjacency assertions and the domain closure axiom to specify the grid structure. The FOL encoding is more readable because each quantified sentence maps directly to the English statement of the rule (e.g., "creaking at `loc` iff damage adjacent to `loc`"), whereas the propositional version repeats the same pattern for every square. Domain closure is needed in FOL because `DeclareSort` creates an open-ended, uninterpreted sort—the solver can invent phantom elements unless told the domain is finite. The propositional encoding needs no such axiom because the set of `Bool` variables is inherently fixed and closed: there are no phantom propositions to worry a